```cs
using Microsoft.EntityFrameworkCore;
using testFinal.Models;

namespace testFinal.Repositories
{
    public class FruitRepository : IFruitRepository
    {
        private readonly ApplicationDbContext _context;

        public FruitRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET ALL
        public async Task<List<Fruit>> GetAll()
        {
            return await _context.Fruits
                .Include(f => f.Category)
                .ToListAsync();
        }

        // GET BY ID
        public async Task<Fruit?> GetById(int id)
        {
            return await _context.Fruits
                .Include(f => f.Category)
                .FirstOrDefaultAsync(f => f.Id == id);
        }

        // ADD
        public async Task Add(Fruit fruit)
        {
            // Validate Category FK
            var categoryExists = await _context.Categories
                .AnyAsync(c => c.Id == fruit.CategoryId);

            if (!categoryExists)
                throw new Exception("Invalid CategoryId");

            // Validate CHECK constraint (Price)
            if (fruit.Price <= 0)
                throw new Exception("Price must be greater than 0");

            _context.Fruits.Add(fruit);
            await _context.SaveChangesAsync();
        }

        // UPDATE
```

```
50          public async Task Update(Fruit fruit)
51          {
52              var existing = await _context.Fruits
53                  .FirstOrDefaultAsync(f => f.Id == fruit.Id);
54
55              if (existing == null)
56                  throw new Exception("Fruit not found");
57
58              // Validate Category FK
59              var categoryExists = await _context.Categories
60                  .AnyAsync(c => c.Id == fruit.CategoryId);
61
62              if (!categoryExists)
63                  throw new Exception("Invalid CategoryId");
64
65              // Update fields (SAFE)
66              existing.FruitsName = fruit.FruitsName;
67              existing.Price = fruit.Price;
68              existing.StockQuantity = fruit.StockQuantity;
69              existing.CategoryId = fruit.CategoryId;
70
71              await _context.SaveChangesAsync();
72          }
73
74          // DELETE
75          public async Task Delete(int id)
76          {
77              var fruit = await _context.Fruits
78                  .FirstOrDefaultAsync(f => f.Id == id);
79
80              if (fruit == null)
81                  return;
82
83              _context.Fruits.Remove(fruit);
84              await _context.SaveChangesAsync();
85          }
86
87          // GET BY CATEGORY
88          public async Task<List<Fruit>> GetByCategoryId(int categoryId)
89          {
90              return await _context.Fruits
91                  .Include(f => f.Category)
92                  .Where(f => f.CategoryId == categoryId)
93                  .ToListAsync();
94          }
95      }
96  }
```